



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 1: ADT 表的实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 18 日

实验报告

【实验目的】

加深对抽象数据类型ADT表的理解；

【实验原理】

利用指针来实现数据的不连续存储

1. 定义链表时多定义一个size变量以储存链表的大小，判断size即可
2. 扫描链表至第*i*个元素，将其输出
3. 删除
 - (1) 将上一个节点的Next指向下一个节点
 - (2) 释放所删除节点的空间
4. 插入（在链表的第*i-1*($1 \leq i \leq n+1$)个结点后插入一个值为X的新结点)
 - (1) 先构造一个新结点，用s指向；
 - (2) 再找到链表的第*i-1*个结点，用p指向；
 - (3) 然后修改指针，插入结点(p之后插入新结点是s)

【实验内容与要求】

编写程序实现ADT表的定义，及常用操作：

- 1) 判断表是否为空；
- 2) 获取第*i*个节点的内容
- 3) 删除
- 4) 插入

【实验源程序】

1.ADT 头文件

```
#ifndef _ADT_H
#define _ADT_H

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* head;
    int size;
} List;

void initList(List* list); //链表初始化
int isEmpty(List* list); //是否非空
int getElement(List* list, int i, int* element); //遍历链表至第 i 个元素
int insertAt(List* list, int i, int data); //插入操作
int deleteAt(List* list, int i); //删除操作
void deleteList(List* list); //删除链表

#endif
```

2.ADT 函数文件

```
#include <stdio.h>
#include <stdlib.h>

/* 链表节点结构体
 * 包含数据域和指向下一个节点的指针 */
typedef struct Node {
    int data; // 节点存储的数据
    struct Node* next; // 指向下一个节点的指针
} Node;

/* 链表结构体
 * 包含链表头指针和记录链表长度的 size */
typedef struct {
    Node* head; // 指向链表第一个节点的指针
```

```

        int size;    // 记录链表当前节点数量
    } List;

/* 初始化链表
 * 将链表头指针置空，size 置 0
 * 参数：list-要初始化的链表指针 */
void initList(List* list) {
    list->head = NULL;
    list->size = 0;
}

/* 判断链表是否为空
 * 参数：list-链表指针
 * 返回值：1 表示空表，0 表示非空表 */
int isEmpty(List* list) {
    return list->size == 0;
}

/* 获取链表中第 i 个元素的值（i 从 1 开始计数）
 * 参数：list-链表指针，i-元素位置，element-用于存储获取值的指针
 * 返回值：1 成功获取，0 获取失败（i 越界） */
int getElement(List* list, int i, int* element) {
    // 检查位置有效性 (1 ≤ i ≤ size)
    if (i < 1 || i > list->size) {
        return 0;
    }

    // 从头节点开始遍历到第 i 个节点
    Node* current = list->head;
    for (int pos = 1; pos < i; pos++) {
        current = current->next;
    }

    *element = current->data; // 通过指针参数返回元素值
    return 1;
}

/* 在链表第 i 个位置插入新节点（i 从 1 开始，允许插入到 size+1 位置）
 * 参数：list-链表指针，i-插入位置，data-插入的数据
 * 返回值：1 插入成功，0 插入失败（i 越界或内存分配失败） */
int insertAt(List* list, int i, int data) {
    // 检查位置有效性 (1 ≤ i ≤ size+1)
    if (i < 1 || i > list->size + 1) {
        return 0;
    }

```

```

    }

    // 创建新节点并分配内存
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        return 0; // 内存分配失败
    }
    newNode->data = data;

    if (i == 1) { // 插入到链表头部
        newNode->next = list->head;
        list->head = newNode;
    } else { // 插入到中间或尾部
        // 找到第 i-1 个节点作为前驱节点
        Node* prev = list->head;
        for (int pos = 1; pos < i - 1; pos++) {
            prev = prev->next;
        }
        // 将新节点插入到前驱节点之后
        newNode->next = prev->next;
        prev->next = newNode;
    }

    list->size++; // 更新链表长度
    return 1;
}

/* 删除链表中第 i 个节点 (i 从 1 开始)
 * 参数: list-链表指针, i-要删除的位置
 * 返回值: 1 删除成功, 0 删除失败 (i 越界) */
int deleteAt(List* list, int i) {
    // 检查位置有效性 (1 ≤ i ≤ size)
    if (i < 1 || i > list->size) {
        return 0;
    }

    Node* temp; // 用于保存要删除的节点指针
    if (i == 1) { // 删除头节点
        temp = list->head;
        list->head = temp->next; // 更新头指针
    } else {
        // 找到第 i-1 个节点作为前驱节点
        Node* prev = list->head;
        for (int pos = 1; pos < i - 1; pos++) {

```

```

        prev = prev->next;
    }
    temp = prev->next;        // 保存要删除的节点
    prev->next = temp->next;  // 前驱节点指向被删节点的下一个
}

free(temp);                // 释放被删除节点的内存
list->size--;               // 更新链表长度
return 1;
}

/* 销毁链表，释放所有节点内存
 * 参数: list-链表指针 */
void deleteList(List* list) {
    Node* current = list->head;
    // 遍历链表逐个释放节点内存
    while (current != NULL) {
        Node* temp = current;    // 保存当前节点指针
        current = current->next;  // 移动到下一个节点
        free(temp);              // 释放当前节点内存
    }
    // 重置链表状态
    list->head = NULL;
    list->size = 0;
}

```

3.主程序应用示例

```

#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"

int main() {
    List list;
    initList(&list);

    // 插入测试
    insertAt(&list, 1, 10); // 链表: 10
    insertAt(&list, 2, 20); // 链表: 10 -> 20
    insertAt(&list, 1, 5);  // 链表: 5 -> 10 -> 20
}

```

```

int elem;
if (getElement(&list, 1, &elem)) {
    printf("第一个元素: %d\n", elem); // 输出 5
}
if (getElement(&list, 2, &elem)) {
    printf("第二个元素: %d\n", elem); // 输出 10
}
if (getElement(&list, 3, &elem)) {
    printf("第三个元素: %d\n", elem); // 输出 20
}

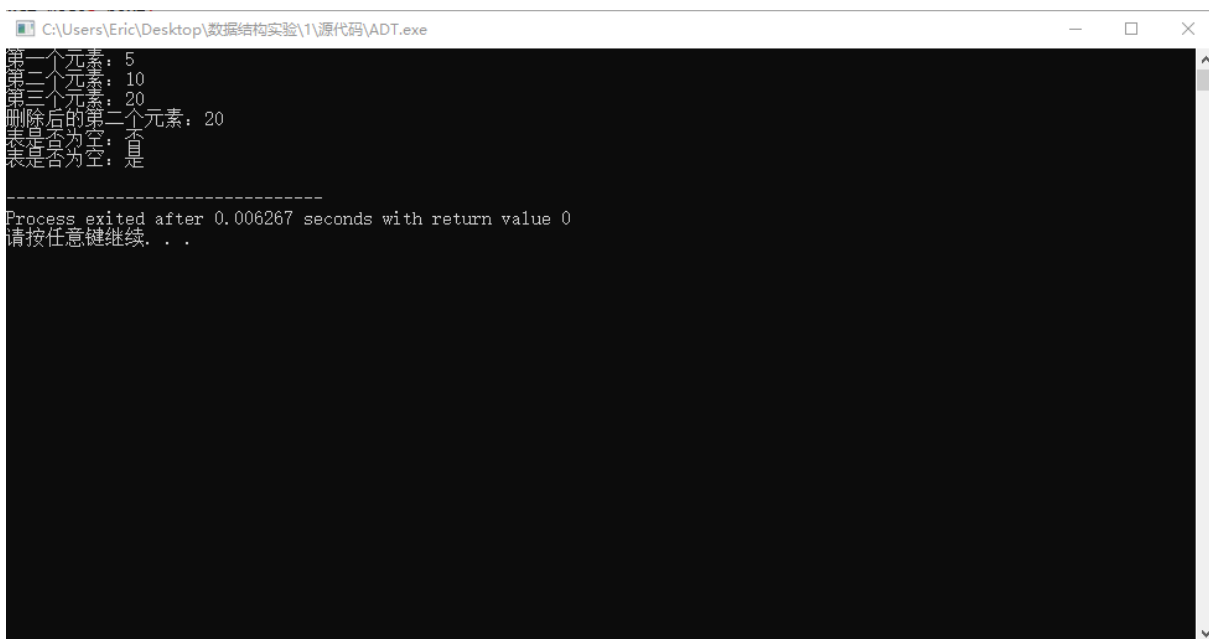
// 删除测试
deleteAt(&list, 2); // 删除第二个元素, 链表: 5 -> 20
if (getElement(&list, 2, &elem)) {
    printf("删除后的第二个元素: %d\n", elem); // 输出 20
}

// 判断空表
printf("表是否为空: %s\n", isEmpty(&list) ? "是" : "否"); // 输出否

// 清空链表
deleteAt(&list, 1);
deleteAt(&list, 1);
printf("表是否为空: %s\n", isEmpty(&list) ? "是" : "否"); // 输出是

deleteList(&list);
return 0;
}
    
```

【实验结果】



```

C:\Users\Eric\Desktop\数据结构实验\1\源代码\ADT.exe
第一个元素: 5
第二个元素: 10
第三个元素: 20
删除后的第二个元素: 20
表是否为空: 否
表是否为空: 是
-----
Process exited after 0.006267 seconds with return value 0
请按任意键继续. . .
    
```

图 1 运行结果截图

可以发现，我们的结果与预期一致，说明我们的 ADT 表构建非常正确，在基础链表操作上，为了防止内存泄露，我们增加了对链表进行删除，初始化操作。

【实验心得】

通过这个实验，我充分地复习了 C 语言有关指针与结构体的知识，才得以完成本次实验。我也遇到了许多困难，其中主要为使用指针访问结构体元素的生疏导致的。

1. 获取第 i 个元素

在实现获取第 i 个元素的函数过程中，我原本想要使用递归方法来进行对第 i 个元素的获取，这样便可以节省代码，如：

```

struct ListNode* get_node_recursive(struct ListNode* head, int index) {
    // 终止条件：空链表或非法索引
    if (head == NULL || index < 0) {
        return NULL;
    }
}
    
```

```
// 终止条件：找到目标节点
if (index == 0) {
    return head;
}

// 递归调用：移动到下一个节点，索引减 1
return get_node_recursive(head->next, index - 1);
}
```

查阅书本后，这样做会大量占用栈的资源，当数据量非常大时，这样的写法非常不稳定，于是放弃，换用迭代完成。

2. 定义链表结构体

在定义链表结构体时，我一开始并没有加上 size 这一变量，但是在完成第二个实验时，我想使用本次实验的函数定义 ADT 队列与 ADT 栈，为了后续实验的方便，我加上了 size 变量。